MIT 8103 Final Summary — Lagos Community Bank Portfolio

Student: Hart Harney
Student ID: 301932250
GitHub: https://github.com/hartharney/Hart_Harney_301932250

1. Case study

I rebuilt this portfolio around **Lagos Community Bank**, a fictional Nigerian retail bank. Tellers and customers need trustworthy balances, atomic fund transfers, statement queries, and an operational event trail. PostgreSQL holds customers, accounts, the ledger, and transfer requests. MongoDB stores append-only `transfer_events` for lifecycle visibility and is demonstrated as a live three-member replica set.

2. Portfolio outcomes

Portfolio 1 — Database design

I normalised a flat teller worksheet from 1NF to 3NF and implemented schema `lcb` with `customers`, `accounts`, `transactions`, `transfers`, `staff`, and `audit_log`. Constraints enforce unique account numbers, non-negative balances, positive transfer amounts, and distinct from/to accounts. Seed data covers dozens of rows; six deliberate bad inserts failed with the expected UNIQUE, CHECK, and FK errors.

Portfolio 2 — Query optimisation

Business SQL joins customers -> accounts -> ledger, produces date-range statements, and aggregates transfer volume by product/branch. After loading **30,000** synthetic ledger rows (30,022 total), an account/date-range statement moved from a sequential scan (**median ~= 1.842 ms**) to a bitmap index scan on `(account_id, posted_at)` (**median ~= 0.201 ms**). `EXPLAIN (ANALYZE, BUFFERS)` showed the filter stopping a near-full heap read.

Portfolio 3 — Transactions and concurrency

A real transfer debited Ada's current account and credited Zainab's savings inside one `BEGIN…COMMIT` with `SELECT FOR UPDATE`. An oversized debit raised and rolled back without changing David's balance. Under READ COMMITTED, Session A re-read Musa's savings and saw Session B's committed +NGN 1,000. A lost-update race without locking finished at **NGN 6,000** (one withdrawal lost); with `SELECT FOR UPDATE` both withdrawals applied and the balance finished at **NGN 3,000**.

Portfolio 4 — NoSQL

MongoDB `transfer_events` documents follow `initiated -> posted/failed` for the same `transfer_ref` values as PostgreSQL. JSON Schema rejected `amount: 0`. Queries covered lifecycle history, failures, posted volume by channel, and latest status per transfer. I treat Mongo as an event store, not a balance authority.

Portfolio 5 — Distributed / cloud (live)

Replica set `lcbRS` ran with members on `127.0.0.1:27017-27019`. I wrote on the primary, read from a secondary (`hello.secondary=true`), then forced `rs.stepDown`. Election promoted `:27018`

to PRIMARY and majority writes resumed (`EVT-RS-AFTER-FAILOVER`, count=4). Majority write concern, replication lag, and sharding trade-offs are discussed; PostgreSQL still needs its own HA for money.

3. Reflection

The previous e-commerce submission taught me that thin samples and discussion-only HA do not convince. Working through bank operations forced me to prove debit+credit atomicity, measure plans on a loaded ledger, and actually elect a new Mongo primary. Progressive commits also changed how I worked: each artefact had to stand alone before the next step began.

4. References

1. PostgreSQL Global Development Group. PostgreSQL 16 Documentation — SQL commands, `EXPLAIN`, concurrency control.

2. MongoDB, Inc. MongoDB Manual — replica sets, write concern, read preference.

3. Connolly, T. & Begg, C. Database Systems — normalisation and transaction concepts (course text).

4. MIT 8103 assignment brief (local `_context/`, not published to GitHub).

5. AI-use declaration

I used **Cursor cloud agents** (Composer) to scaffold SQL/JS scripts, run PostgreSQL and MongoDB locally, capture evidence, and keep progressive Git commits moving. I directed the Lagos Community Bank case study, reviewed every script against the assignment criteria, executed the database demos myself in this environment, and wrote the technical narrative in first person. All runtime evidence (plans, dual-session logs, replica failover) comes from live commands, not invented screenshots. I did not paste the prior e-commerce wording into this rebuild.